DATABASE DESIGN

For AI-Enabled Web Final Projects

Student / Group	Project Title	Date
S. Kajarsan	Travelly	21 October 2026 Database Target: PostgreSQL 16 with PostGIS

Step 1 - Review Your Use Cases and Identify Data

For every important Travelly use case, the following table identifies the data that the system must read, create, update or store. AI search is included as a processing use case, but raw AI conversation history is intentionally excluded from the NBP.

Use Case	Data Needed	Possible Entity / Tables
Register and sign in	name, email/phone, password hash, verification, account status	users
Apply for roles	Passenger/Owner/Driver/Admin role and approval state	user_roles
Verify user or vehicle documents	document type, number, issue/expiry dates, secure file and review result	documents
Register and publish vehicle	Owner, registration, type, make/model, power, seats, luggage, odometer, media	vehicles
Offer hire/ticket/rental service	service category, pricing basis, price and Owner terms	service_types; vehicle_services
Maintain route and stops	origin, destination, ordered pickup/drop stops, distance, duration	routes; route_stops
Publish Ticket departure	route, vehicle, Owner, time, fare, cutoff, fulfilment policy and deadline	scheduled_trips
Create 2D seat layout	seat code, row, column, deck, type and bookable state	vehicle_seats
Hold/book a departure seat	departure, seat, status, hold expiry and booking	scheduled_trip_seats
Book private hire, Ticket or rental	Passenger, service, itinerary, capacity, luggage, vehicle/departure, price, status	bookings
Record travellers	name, contact, seat, boarding/drop stop and assistance	booking_travellers

Operate a trip	Owner self-drive or assigned Driver, acceptance, status, odometer and current location	trips
Pay/refund/reallocate/payout	booking, transaction type, provider, amount, commission and status	payments
Maintain vehicle and block availability	oil, tyres, alignment, service, fuel/charging, odometer, personal use, unavailable range	vehicle_records
Notify and reconfirm	recipient, booking/trip, channel, delivery/read and action response; IVR response fields are reserved for a pending optional future integration	notifications
Post Travelly Reel/photo/review	author, media, caption, location and linked vehicle/route/departure	social_posts
Like/comment/save/share/follow/report/rate	actor, target, content/rating/report and status	social_interactions
Report active-route road alert	type, active trip, coordinates, direction, confirmations, expiry, moderation	road_alerts
AI text/voice search	temporary extracted filters; query actual eligible inventory	No AI table in NBP; reads routes, scheduled_trips, vehicles, services and availability
Admin management	approvals, services, routes, bookings, payments, replacements and moderation	Uses the same domain tables

Step 2 - Identify Entities

An entity is a real-world object or business concept for which Travelly must store persistent data. The candidate entities were reduced to the following final 20 tables so the one-month NBP remains buildable while preserving the full Passenger–Owner–Driver–Admin flow.

Identity and verification

- users
- user_roles
- documents

Fleet and service catalogue

- vehicles
- service_types
- vehicle_services

Routes, Ticket departures and seats

- routes
- route_stops
- scheduled_trips
- vehicle_seats
- scheduled_trip_seats

Booking, operation and finance

- bookings
- booking_travellers
- trips
- payments
- vehicle_records
- notifications

Travelly community and route safety

- social_posts
- social_interactions
- road_alerts

A Passenger, Owner and Driver are not stored as three duplicate people. They share one users record and receive separate role rows in user_roles. Passenger access is created for the shared account; Owner and Driver applications are approved independently. An eligible Owner may choose self-drive inside the Owner dashboard without a second login, while an external Driver assignment remains pending until that Driver accepts or rejects. This supports one account with multiple roles while keeping approval and suspension independent.

Step 3 - Define Attributes for Each Entity

Each row below summarises the important attributes of one entity. Core relationships remain UUID foreign keys. JSONB is used only for bounded settings, media metadata and accepted policy snapshots; it does not replace relational keys.

Entity	Important Attributes	Main Data Types	Required?	Notes / Rule
users	id, email/phone, password_hash, full_name, profile, account_status, verification timestamps	UUID, CITEXT, VARCHAR, TEXT, TIMESTAMPTZ	id/name/status/password; verified contact before booking	one identity for all roles

user_roles	id, user_id, role_code, approval_status, approved_by/at, rejection/suspension	UUID, VARCHAR, TIMESTAMPTZ, TEXT	user/role/status	UNIQUE user_id + role_code
documents	user_id or vehicle_id, type/number, dates, storage metadata, status, verifier	UUID, VARCHAR, DATE, TEXT, BIGINT, TIMESTAMPTZ	exactly one owner target; type/file/status	retain superseded history
vehicles	Owner, registration, type, make/model, power/fuel, seats/luggage, odometer, media, statuses	UUID, CITEXT, VARCHAR, SMALLINT, NUMERIC, JSONB, BOOLEAN	Owner/registration/capacity/status	BEV fuel compatibility; registration unique
service_types	code, name, category, pricing basis, active, settings	UUID, CITEXT, VARCHAR, BOOLEAN, JSONB	code/name/category	Admin-managed definitions
vehicle_services	vehicle, service, enabled, pricing values, currency, availability, terms	UUID, BOOLEAN, NUMERIC, CHAR, DATE, JSONB	vehicle/service/enabled	UNIQUE vehicle + service
routes	code/name, origin/destination, coordinates, distance/duration, status, approval	UUID, CITEXT, VARCHAR, NUMERIC, INTEGER, TIMESTAMPTZ	code/origin/destination/status	PostGIS geography optional
route_stops	route, order, code/name, coordinates, pickup/drop flags, time offset	UUID, SMALLINT, VARCHAR, NUMERIC, BOOLEAN, INTEGER	route/order/name	unique order per route
scheduled_trips	Owner/service/route/vehicles/Driver, times, fare, policy/minimum/deadline, status/replacement	UUID, TIMESTAMPTZ, NUMERIC, CHAR, VARCHAR, JSONB	Owner/route/vehicle/time/fare/policy/status	snapshot accepted protection terms
vehicle_seats	vehicle, layout version, seat code, row/column/deck, type, active/bookable	UUID, INTEGER, SMALLINT, VARCHAR, BOOLEAN, JSONB	vehicle/version/code/position	unique coordinate in layout
scheduled_trip_seats	departure, vehicle seat, status, hold booking/expiry, confirmed booking, version	UUID, VARCHAR, TIMESTAMPTZ, NUMERIC, INTEGER	departure/seat/status	transaction lock prevents double booking
bookings	Passenger/type/service/Owner/vehicle/departure, itinerary/time, counts/luggage, status/assurance, money/cancellation/idempotency	UUID, VARCHAR, TIMESTAMPTZ, SMALLINT, NUMERIC, CHAR, JSONB	Passenger/type/service/time/status/currency	original and current fulfilment retained
booking_travellers	booking, linked user, name/contact, seat, boarding/drop stop, assistance/status	UUID, VARCHAR, TEXT, BOOLEAN	booking/name/status	one active traveller per booked seat
trips	booking/departure, vehicle/Owner/Driver, assignment/status, times, odometer/distance, current location	UUID, VARCHAR, TIMESTAMPTZ, NUMERIC, TEXT	vehicle/Owner/assignment/status	GPS only during active trip
payments	booking/original payment/payee, transaction/method/provider/idempotency, amounts/currency/status/timestamps	UUID, VARCHAR, NUMERIC, CHAR, TIMESTAMPTZ	booking/type/amount/currency/status	refund/reallocation/payout are new rows

vehicle_records	vehicle/trip/recorder, record type/time/odometer, cost/quantity/energy, due km/date, receipt/notes	UUID, VARCHAR, TIMESTAMPTZ, NUMERIC, DATE, TEXT	vehicle/type/time/recorder	typed rows also store availability blocks
notifications	recipient and optional booking/departure/trip, type/channel/content/action/status/response/expiry; current channels are in-app/email and IVR remains pending	UUID, VARCHAR, TEXT, BOOLEAN, JSONB, TIMESTAMPTZ	recipient/type/channel/status	no-response alone cannot cancel
social_posts	author/type/media/caption/hashtags/location, optional vehicle/route/departure, visibility/moderation/counts	UUID, VARCHAR, TEXT, JSONB, NUMERIC, TIMESTAMPTZ	author/type/media/moderation	media bytes stored outside DB
social_interactions	actor, post or target user, type, comment/rating/report/status, parent	UUID, VARCHAR, TEXT, SMALLINT, JSONB	actor/type and valid target	partial uniqueness for like/save/follow
road_alerts	reporter/trip, type, location/direction, source/status/confidence/counts/expiry/moderator	UUID, VARCHAR, NUMERIC, INTEGER, TIMESTAMPTZ, TEXT	reporter/type/location/status/expiry	route-aware and automatically expires

Step 4 - Select Primary Keys

Travelly uses id UUID as the immutable primary key of every application table. UUIDs are suitable for public APIs, distributed creation and non-sequential identifiers. Human-readable references such as booking_reference, departure_reference, trip_reference and payment_reference are separate UNIQUE business identifiers.

☑ Every entity has one primary key.
☑ Primary key values are unique and never reused.
☑ Primary keys do not change during the record lifetime.
☑ Composite UNIQUE constraints enforce business identity where needed.

Important alternate keys: users(email), users(phone_e164), vehicles(registration_number), service_types(code), routes(route_code), user_roles(user_id + role_code), vehicle_services(vehicle_id + service_type_id), route_stops(route_id + stop_order), vehicle_seats(vehicle + layout version + seat code), scheduled_trip_seats(departure + seat), and payment/booking idempotency keys.

Step 5 - Identify Relationships

Relationship cardinality describes how many records on one side may connect to the other. Many-to-many relationships are resolved through junction tables.

Relationship Type	Meaning	Travelly Example
1 : 1	One record relates to at most one operational record	A confirmed booking normally creates one trip

1 : Many	One parent relates to many children	One Owner has many vehicles
Many : Many	Many records relate through a junction table	Vehicles and service types through vehicle_services

Travelly project relationships

Entity A	Relationship	Entity B	Cardinality	Business Rule
users	has	user_roles	1:M	one account may hold several roles; each role only once
users / vehicles	owns	documents	1:M	each document belongs to exactly one user or vehicle
users (Owner)	owns	vehicles	1:M	one Owner may maintain multiple vehicles
vehicles	offers through vehicle_services	service_types	M:N	only enabled approved combinations appear
routes	contains	route_stops	1:M	stops have a unique sequence
routes	schedules	scheduled_trips	1:M	each departure uses one route
vehicles	defines	vehicle_seats	1:M	layout is versioned
scheduled_trips	instantiates	scheduled_trip_seats	1:M	one inventory row per seat
vehicle_seats	is used by	scheduled_trip_seats	1:M	layout seat can appear in many departures
users (Passenger)	creates	bookings	1:M	Passenger sees only own bookings
bookings	contains	booking_travellers	1:M	Ticket seats attach to travellers
bookings	has	payments	1:M	refund/reallocation/payout are related events
bookings / scheduled_trips	produces	trips	1:0..1	normal NBP creates one operational trip
users (Driver)	operates	trips	1:M	Owner may be same eligible Driver
vehicles	runs	trips	1:M	time conflicts prohibited
vehicles	has	vehicle_records	1:M	maintenance, energy, odometer and

				blocks
users	receives	notifications	1:M	delivery/response retained
users	authors	social_posts	1:M	public content subject to moderation
social_posts	has	social_interactions	1:M	likes/comments/saves/shares/reports/ratings
trips	may contain	road_alerts	1:M	alerts exist only until expiry

Step 6 - Add Foreign Keys

A foreign key stores the parent primary key in the related child table. Travelly uses foreign keys to enforce identity, ownership, fleet, Ticket inventory, booking, operation, payment and content relationships.
Main foreign-key paths:
• users → user_roles, vehicles, bookings, trips, notifications, social_posts, social_interactions and road_alerts.
• users or vehicles → documents; exactly one owner target is allowed.
• vehicles + service_types → vehicle_services.
• routes → route_stops and scheduled_trips.
• vehicles → vehicle_seats, scheduled_trips, trips and vehicle_records.
• scheduled_trips + vehicle_seats → scheduled_trip_seats.
• bookings → booking_travellers, trips, payments and notifications.
• social_posts → social_interactions; trips → road_alerts.
☑ Every 1:M relationship has the correct foreign key.
☑ FK data types match the referenced UUID primary keys.
☑ Required relationships use NOT NULL.
☑ Completed booking, trip and payment history uses RESTRICT/archive status instead of destructive deletion.
☑ Draft-only child records may cascade only where no legal or audit history is lost.

Step 7 - Draw the ER Diagram (ERD)

The ER diagram below shows all 20 tables as one connected model. Each entity displays its primary key and important foreign keys. Arrows point from parent to child, and relationship labels show cardinality.

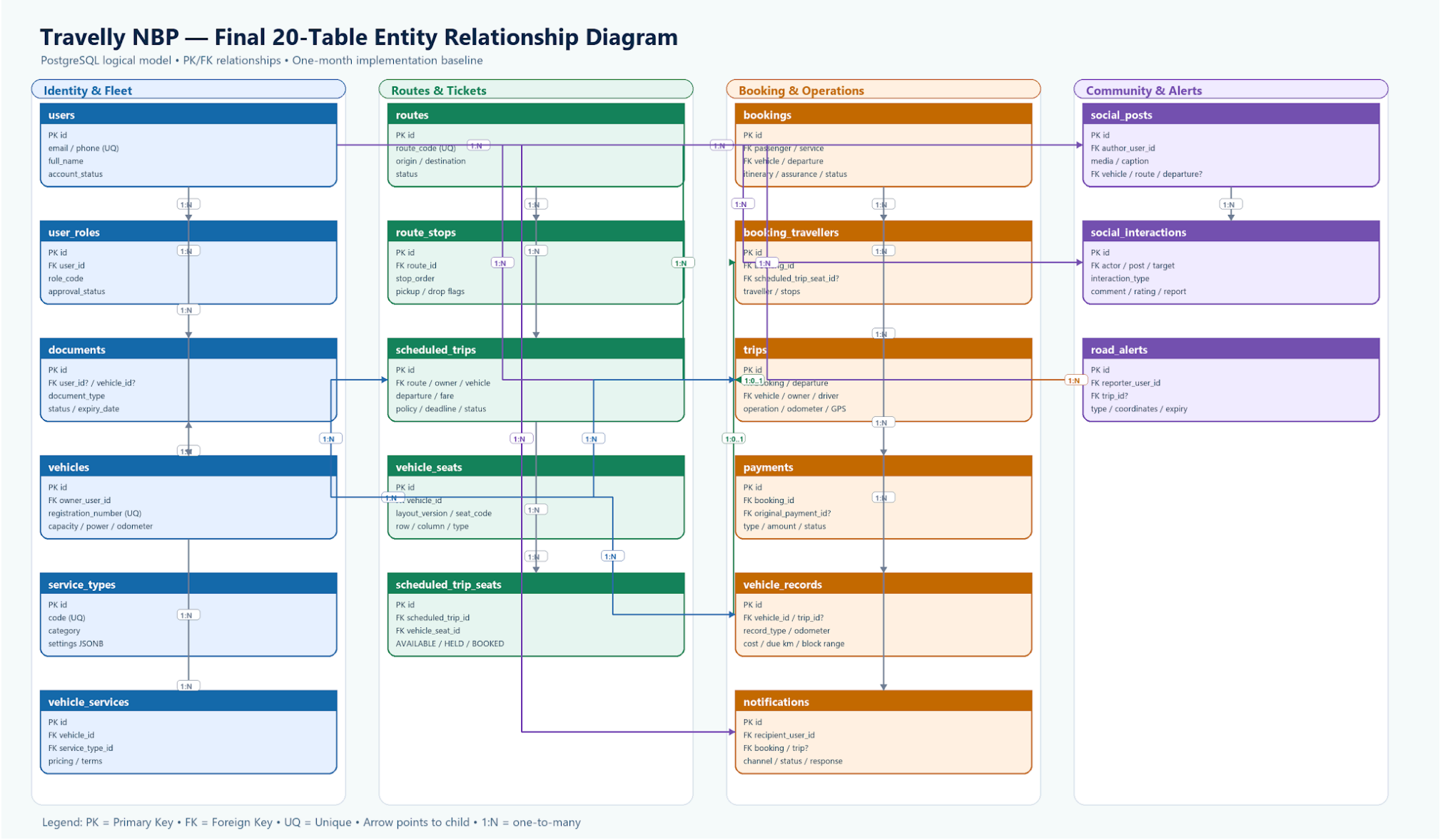


Figure 1. Travelly NBP final 20-table entity relationship diagram.
The diagram is the visual implementation reference. The detailed field definitions and constraints remain in Step 10.

Step 8 - Normalise the Database

The database is designed to Third Normal Form (3NF). Controlled historical snapshots are retained only where later configuration changes must not alter Passenger terms already accepted.

Level	Travelly Check	Result
1NF	Roles, services, stops, travellers, seats, payments, records and interactions are separate rows; core fields are atomic.	Passed
2NF	Attributes in junction tables describe the complete relationship, e.g. vehicle-service pricing and departure-seat state.	Passed
3NF	Service definitions, routes/stops, layout/inventory, booking/trip and payment states are separated; non-key fields depend on their PK.	Passed
Controlled snapshots	protected_terms_snapshot and cancellation_policy_snapshot intentionally preserve accepted historical terms.	Approved denormalisation

Step 9 - Design AI-Related Data Carefully

Travelly uses AI to understand a Passenger's text/voice search and to classify a spoken road alert. AI does not create inventory, prices or guaranteed facts. The backend validates extracted values and queries actual routes, vehicles, services, departures and availability.

AI search process:

1. Receive text or speech.
2. Extract origin, destination, date/time, passengers, service intent, luggage and explicit preferences.
3. Ask the Passenger to confirm uncertain fields.
4. Query real eligible inventory.
5. Store only the resulting normal booking if the Passenger proceeds.

AI Data Question	Travelly Decision	Stored Where
Store raw user question?	No for NBP; process transiently and confirm uncertain fields.	Not stored
Store AI response?	No; AI suggestion is not inventory or guaranteed truth.	Not stored
Store extracted search filters?	Only as normal booking fields if the user proceeds.	bookings
Store AI-created vehicle/price?	Never. Backend returns real eligible records only.	vehicles, vehicle_services, scheduled_trips

Store voice road alert?	Store validated structured alert, not raw audio/transcript.	road_alerts
Store AI calling?	No conversational AI calling in the NBP. Telephony/IVR is pending and not implemented now; if later required and approved, the existing notification response structure may record authenticated Press 1/Press 2 outcomes.	notifications
Privacy rule	Avoid unnecessary personal, voice and sensitive data; use approved provider retention settings.	Infrastructure policy

☑ Raw voice and AI conversation history are not unnecessarily stored.
☑ AI results are not treated as database truth.
☑ Conversational AI calling is excluded.
☑ Telephony/IVR remains pending and unimplemented; notification response fields preserve an optional future Press 1/Press 2 extension without adding a table.
☑ Provider privacy and retention settings must be reviewed before release.

Step 10 - Create the Final Relational Schema

The following native table is the backend reference for the one-month NBP. PK = Primary Key, FK = Foreign Key, UQ = Unique and NN = Not Null.

Table	Core Column(s)	Type	PK	FK	Null?	Constraint / Description
users	id; email; phone; password_hash; full_name; account_status; timestamps	UUID/CITEXT/VARCHAR/TEXT/TIMESTAMPTZ	id	—	email or phone may be null individually	unique contacts; shared identity
user_roles	id; user_id; role_code; approval_status; approved_by/at	UUID/VARCHAR/TIMESTAMPTZ	id	user_id, approved_by	approval fields conditional	UNIQUE user+role
documents	id; user_id/vehicle_id; type/number; dates; storage; status; verifier	UUID/VARCHAR/DATE/TEXT/TIMESTAMPTZ	id	user, vehicle, verifier, superseded doc	one owner target only	CHECK XOR owner; keep history
vehicles	id; Owner; registration; type/make/model/power; seats/luggage; odometer; media; statuses	UUID/CITEXT/VARCHAR/SMALLINT/NUMERIC/JSONB	id	owner, default Driver	default Driver optional	registration UQ; capacity non-negative
service_types	id; code; name; category; pricing basis; active; settings	UUID/CITEXT/VARCHAR/BOOLEAN/JSONB	id	—	description optional	Admin catalogue; code UQ

vehicle_services	id; vehicle; service; enabled; prices; currency; dates; terms	UUID/BOOLEAN/NUMERIC/CHAR/DATE/JSONB	id	vehicle, service	optional price components	UNIQUE vehicle+service
routes	id; code/name; origin/destination; coordinates; distance/duration; status; approval	UUID/CITEXT/VARCHAR/NUMERIC/INTEGER	id	creator/approver users	geometry optional	route code UQ
route_stops	id; route; order; code/name; coordinates; pickup/drop; offset	UUID/SMALLINT/VARCHAR/NUMERIC/BOOLEAN	id	route	code optional	UNIQUE route+order
scheduled_trips	id; reference; Owner/service/route/vehicles/Driver; times/fare; policy/minimum/deadline; statuses	UUID/VARCHAR/TIMESTAMPTZ/NUMERIC/JSONB	id	users, service, route, vehicles, replacement trip	operator/replacement optional	accepted terms frozen
vehicle_seats	id; vehicle; layout version; seat code; row/column/deck; type; flags	UUID/INTEGER/SMALLINT/VARCHAR/BOOLEAN	id	vehicle	metadata optional	unique code/coordinate per layout
scheduled_trip_seats	id; departure; seat; status; hold booking/expiry; booked booking; version	UUID/VARCHAR/TIMESTAMPTZ/INTEGER	id	departure, vehicle seat, bookings	hold/booking conditional	UNIQUE departure+seat; lock atomically
bookings	id; reference; Passenger/type/service/Owner/vehicle/departures/stops; time/counts; assurance; money/status	UUID/VARCHAR/TIMESTAMPTZ/SMALLINT/NUMERIC/JSONB	id	users, service, vehicle, departures, stops	depends on booking type	idempotency UQ; original/current retained
booking_travellers	id; booking; linked user; name/contact; seat; stops; assistance; status	UUID/VARCHAR/TEXT/BOOLEAN	id	booking, user, seat, stops	linked user/seat optional	one active traveller per seat
trips	id; reference; booking/departure; vehicle/Owner/Driver; assignment/status; times; odometer; current GPS	UUID/VARCHAR/TIMESTAMPTZ/NUMERIC/TEXT	id	booking, departure, vehicle, users	booking/departure/Driver conditional	GPS only active trip
payments	id; reference; booking/original payment/payee; type/method/provider; idempotency; amounts/status/times	UUID/VARCHAR/NUMERIC/CHAR/TIMESTAMPTZ	id	booking, payment, user	payee/refund times conditional	append-only; idempotency UQ
vehicle_records	id; vehicle/trip/recorder;	UUID/VARCHAR/TIMESTA	id	vehicle, trip, user	type-specific fields optional	typed

	type/time/odometer; block range; cost/energy; due km/date; receipt	MPTZ/NUMERIC/DATE/TEXT				maintenance/expense/block records
notifications	id; recipient; context; type/channel/content/action/status; delivery/response/expiry	UUID/VARCHAR/TEXT/BOOLEAN/JSONB/TIMESTAMPTZ	id	user, booking, departure, trip	context optional	no-response cannot auto-cancel
social_posts	id; author/type/media/caption/hashtags/location; optional links; visibility/moderation/counts	UUID/VARCHAR/TEXT/JSONB/NUMERIC	id	user, vehicle, route, departure	links optional	media outside DB
social_interactions	id; actor; post/target; type; comment/rating/report/status; parent	UUID/VARCHAR/TEXT/SMALLINT/JSONB	id	users, post, parent	fields depend on type	partial UQ like/save/follow
road_alerts	id; reporter/trip; type/location/direction; source/status/confidence/counts/expiry; moderator	UUID/VARCHAR/NUMERIC/INTEGER/TIMESTAMPTZ	id	users, trip	trip/direction/moderator optional	geospatial index; auto-expiry

Additional Travelly Data Not Stored as Separate NBP Tables

Handled without adding another application table

- Vehicle/document/Reel media files are stored in object storage or a CDN; the database stores URLs and metadata.
- Search results and vehicle availability are calculated from services, schedules, bookings, trips and availability blocks.
- Dashboard totals and PDF reports are generated from bookings, trips, payments and vehicle_records.
- Maintenance remaining kilometres and estimated due dates are calculated from the current odometer and recorded usage.
- The current/last-known active-trip location is stored on trips; a full GPS-history table is not included.
- AI search is processed statelessly; a separate AIQuery table is not required.
- Authentication sessions, cache, queues, media processing and system logs belong to the selected infrastructure services.

Phase 2 data that will require future schema expansion

- Subscription plans, subscription billing and secondary-owner delegation.
- Direct chat conversations, messages and attachments.
- Conversational AI calls, transcripts and consent records.

• Route-segment seat reuse and automatic demand aggregation.
• Automatic cross-Owner settlement, reserves, compensation and full accounting ledger.
• Rental handover/return inspections, deposits and damage claims.
• Full GPS breadcrumb history and telematics.
• Advanced audit-event and Admin configuration versioning.
• Advanced Reels watch history, recommendation data, music, stories, live streaming and monetisation.
• Promotions, coupons, loyalty and referral ledger.
• Predictive maintenance, diagnostics and charging-network integration.
These are deliberate Phase 2 exclusions, not missing NBP tables.

Implementation Baseline and Integrity Rules

• Online Ticket seats require full online payment before confirmation; cash/manual Ticket sales remain Phase 2.
• NBP cash is limited to enabled private-hire/basic-rental services, with Owner-selected start/end collection timing and Passenger plus collecting Driver/self-driving Owner confirmation.
• Only the assigned active Driver or eligible self-driving Owner may create a road alert. Passengers may receive and submit feedback.
• Admin configuration is bounded to service-specific settings and per-vehicle thresholds in the approved 20-table schema; a global rules engine is Phase 2.
• Media files remain in protected object storage/CDN while the database stores authorized URLs and metadata.

Final Review

The completed design contains exactly 20 application tables. It covers shared accounts, multiple roles, private hire, basic rental, route and airport Travelly Protected Ticket Hire, 2D seats, Owner self-drive, Driver assignment, availability blocking, payment/refund/reallocation, vehicle maintenance, notifications, Reels, AI-assisted search and role-restricted road alerts. It is ready for PostgreSQL migrations and implementation against the aligned API specification.